

5 PROJETO E ANÁLISE DE ALGORITMOS

Vamos examinar estratégias para projeto de algoritmos e sua complexidade.

Problema:

- Estrutura interna aproveitada na sua resolução.
- Relacionamento entre problemas:
 - caso especial;
 - versão abstração;
 - similar.

Exemplos:

Encontrar raízes complexas de polinômios quadráticos com coeficientes reais é um **caso especial** (bem mais simples) do problema de encontrar raízes complexas de polinômios de grau arbitrário com coeficientes reais.

Inserir um objeto em uma estrutura é uma **versão abstrata** do problema de inserir um elemento em uma lista.

A busca de um elemento em uma tabela ou em uma árvore são ditos **similares**.

Idéia: analisar famílias de problemas, cuja solução é alcançada de forma semelhante.

5.1 MÉTODO GULOSO

Vamos examinar algoritmos chamados gulosos. A **estratégia gulosa** é útil principalmente para resolver **problemas de otimização combinatória**, cuja solução pode ser alcançada por uma seqüência de decisões.

A estratégia gulosa pode ser aplicada a problemas de otimização combinatória, como por exemplo:

- intercalação sucessiva ótima de listas;
- caminhos de custo mínimo em grafos orientados;
- árvore geradora (ou de espalhamento) mínima em grafos não orientados;
- códigos para transmissão de mensagens (códigos de Huffman);
- escalonamento de tarefas.

Exemplo introdutório

Um exemplo simples que ilustra bem a estratégia gulosa é o problema da **intercalação sucessiva de listas**. Trata-se de intercalar n listas. Há várias maneiras possíveis de se realizar uma seqüência de intercalações. Ocorre que isso pode afetar o número de comparações necessárias.

Intercalação de Listas.

O número de comparações necessárias para intercalar duas listas, no **pior caso**, é a soma dos seus comprimentos, que também é o comprimento da lista resultante, menos 1.

Sejam três listas L_1 , L_2 e L_3 , com respectivos comprimentos 15, 10 e 5.

Uma maneira de proceder seria: primeiro intercalar L_1 com L_2 e, depois, o resultado $_1L_2$ com L_3

- O número de comparações é $(15+10-1)$, para produzir $_1L_2$, e $(15+10)+5-1$, para intercalá-la com L_3 .

- No total, temos $(15+10-1) + (15+10)+5-1$, ou seja, 53 comparações.

Uma **outra maneira** de proceder é: primeiro intercalar L_2 com L_3 e, depois L_1 com o resultado $_2L_3$.

O número de comparações é $(10+5-1)$, para produzir $_2L_3$, e $15+(10+5-1)$, para intercalar L_1 com $_2L_3$.

No total, temos $(10+5-1) + 15+(10+5-1)$, ou seja 43 comparações.

Então ordem em que são realizadas as intercalações pode influir substancialmente no **número de comparações** requeridas.

Agora vamos examinar mais de perto o efeito da **seqüência escolhida** no número de comparações requeridas para intercalar n listas L_i com comprimentos m_i , para $i = 1, \dots, n$.

Consideremos, no caso de $n = 3$ listas, as duas maneiras acima:

- na primeira maneira, o número total t' de comparações é $_1m_2 + (_1m_2 + m_3)$, ou seja, $2 \cdot (m_1 + m_2) + m_3$.
- na segunda maneira, número total t'' de comparações é $_2m_3 + (m_1 + _2m_3)$, ou seja, $2 \cdot (m_2 + m_3) + m_1$.

Se $_1m_2 \leq _2m_3$ i. e., $(m_1 + m_2) \leq (m_2 + m_3)$, teremos $t' \leq t''$; nesse caso, devemos escolher a primeira maneira.

É isso que sugere a **estratégia gulosa**: escolher o que dá menor contribuição ao resultado final.

De fato, consideremos intercalar as n listas L_i nessa ordem.

1. Inicialmente, intercalamos L_1 com L_2 , usando ${}_1m_2$ comparações e produzindo ${}_1L_2$ com comprimento ${}_1m_2$, onde ${}_1m_2 := (m_1 + m_2)$
- i. No passo i , intercalamos ${}_iL_i$ com L_{i+1} , usando ${}_im_{i+1}$ comparações e produzindo ${}_iL_{i+1}$ com comprimento ${}_im_{i+1}$, onde ${}_im_{i+1} := ({}_im_i + m_{i+1})$
- n-1. Finalmente, intercalamos ${}_nL_{n-1}$ com L_n , usando ${}_nm_n := ({}_nm_{n-1} + m_n)$ comparações

O número total de comparações é dado por ${}_1m_2 + \dots + {}_im_{i+1} + \dots + {}_nm_n$, ou seja,

$$(n-1) \cdot m_1 + (n-1) \cdot m_2 + (n-2) \cdot m_3 + \dots + (n-i) \cdot m_{i+1} + \dots + m_n - (n-1)$$

O primeiro par de listas a ser intercalado dá contribuição maior para o peso total.

Consideremos **5 listas**, com comprimentos 10, 20, 30, 40 e 50, a intercalar:

1. Inicialmente, escolhemos as duas de menores comprimentos (10 e 20) ficando com 4 listas, com comprimentos 30, 30, 40 e 50.
2. Repetindo o processo, escolhemos as de comprimentos 30 e 30 ficando com 3 listas, com comprimentos 60, 40 e 50.
3. Agora, escolhemos as de comprimentos 40 e 50, ficando com 2 listas, com comprimentos 60, 90.
4. Finalmente, substituímos essas duas listas pelo resultado de sua intercalação, ficando com uma lista: o resultado final.

10	20	30	40	50			
1	2	3					
1	4	3					
	2	60	1	2	3		
1	4	4	4	2	4	3	90
				4	150		

Algoritmo : Intercalação ótima de listas

Função Interc_Suc_Lst($L : D$) $\rightarrow R$ {intercalação ótima de listas}

{ *Entrada*: multiconjunto $L : D$ de n listas (com seus comprimentos) a intercalar

Saída: par (intercalação das n listas em L , número mínimo de comparações) }

sorte $D :=$ multiconjunto de $\mathbb{N}$ {comprimentos de listas} ;

var {auxiliares} $ncp : \mathbb{N}$; $L^*, L', L'' : \text{Listas}$;

1. $ncp \leftarrow 0$; {inicializa saída}
2. repita {itera até $|L| = 1:8$ }
3. escolhe as duas menores listas $L' \text{ e } L''$ em L ; {seleção gulosa}
4. $L \leftarrow L - \{ L', L'' \}$; {remove elementos selecionados}
5. $L^* \leftarrow \text{Interc}(L', L'')$; { L^* : intercalação de listas selecionadas}
6. $L \leftarrow L \cup \{L^*\}$; {inclui resultado da intercalação}
7. $ncp \leftarrow ncp + \text{tam}(L') + \text{tam}(L'') - 1$; {atualiza solução}
8. até-que $|L| = 1$; {2: uma única lista}
9. $L^* \leftarrow$ a única lista em L ; {recupera lista encadeada}
10. retorne-saída (L^*, ncp) ; {dá saída recuperada}
11. fim-Função {fim do algoritmo Interc_Suc_Lst}

Idéias básicas

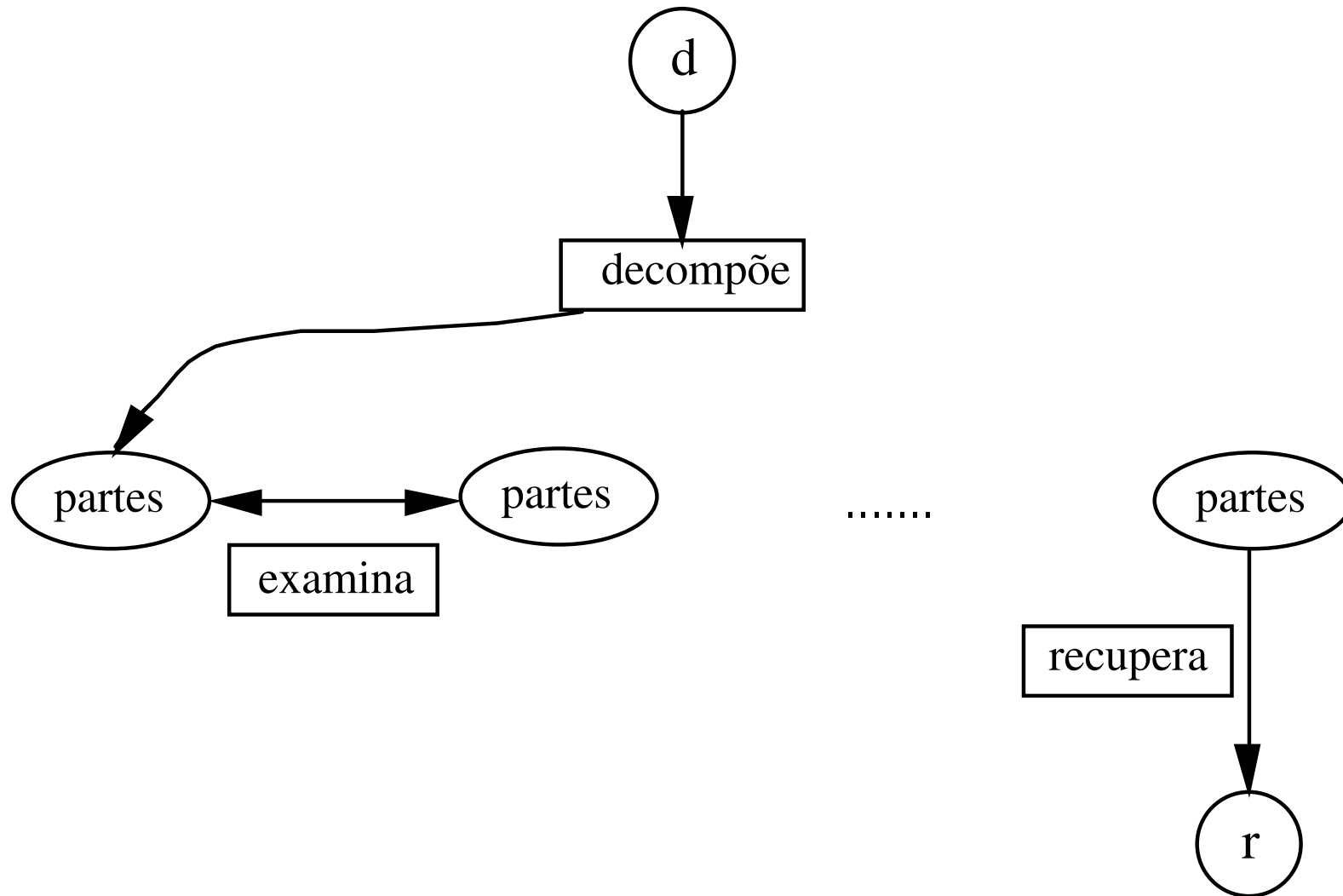
Um algoritmo guloso para seqüência ótima de intercalações seleciona, a cada passo um par com **soma mínima** dentre os presentes para fazer parte da solução.

Em cada passo:

- seleciona um elemento e decide-se se ele é viável - vindo a fazer parte da solução - ou não.
- após uma seqüência de decisões, a solução do problema é alcançada.

Na **seqüência de decisões**, nenhum elemento é examinado mais de uma vez: ou ele fará parte da saída, ou será descartado.

O diagrama a seguir mostra a estrutura geral dessa abordagem.



Estrutura geral do método guloso

A idéia básica da **estratégia gulosa** sugere uma estrutura geral para algoritmos gulosos: uma seqüência, consistindo de um trecho de inicialização, seguido de uma iteração e um trecho de finalização.

Assim, um algoritmo guloso tem a forma

Inicialização ; Iteração ; Finalização.

A **inicialização** prepara a entrada (muitas vezes a entrada é classificada) e inicializa a saída.

A **iteração** seleciona um elemento conforme uma função gulosa, marca-o para não considerá-lo novamente no futuro, atualiza a entrada, examina o elemento selecionado quanto sua viabilidade e decide a sua participação ou não na solução.

A **finalização** recupera a saída.

Dessa maneira, pode-se chegar ao seguinte esquema geral de algoritmo guloso.

A estrutura algorítmica do método guloso é dada pelo seguinte programa abstrato.

Algoritmo : Algoritmo Guloso

Função Alg_Gul($d:D$) $\rightarrow R$ {Algoritmo Guloso (abstrato)}

{ *Entrada-saída*: saída $r:R$ é resposta ótima para entrada $d:D$ }

Inc_G ; Iter_G ; Fnl_G {estrutura geral}

onde {componentes}

Inc_G: {inicialização}

inicializa parte da entrada a examinar e resposta parcial

Iter_G: itera corpo Crp_G {iteração}

seleciona elemento e da parte da entrada a examinar;

remove elemento e da parte da entrada a examinar;

inclui elemento e na resposta parcial se viável

Fnl_G: {finalização}

recupera a saída a partir da resposta parcial

No algoritmo Interc_Suc_Lst (para intercalação ótima de listas) apresentado no exemplo, podemos estabelecer as seguintes identificações:

Interc_Suc_Lst	Alg_Gul
<u>sorte</u> D := multiconjunto de listas	<u>sorte</u> D {entrada}
1. $ncp \leftarrow 0$;	IncZ_G {inicialização}
2. <u>repita</u>	Iter_G {iteração}
3. $(L', L'') \leftarrow 2 \text{ menores em } L$;	seleciona e da parte da entrada;
4. $L \leftarrow L - \{L', L''\}$;	
5. $L^* \leftarrow \text{Interc}(L', L'')$;	remove e da parte da entrada;
6. $L \leftarrow L \cup \{L^*\}$;	
7. $ncp \leftarrow ncp + \text{tam}(L') + \text{tam}(L'') - 1$;	inclui e na resposta parcial se viável;
8. <u>até-que</u> $ L = 1$;	{fim da iteração}
9. $L^* \leftarrow \text{a única lista em } L$;	FnI_G {finalização}

Tabela : Identificações entre Interc_Suc_Lst e Alg_Gul

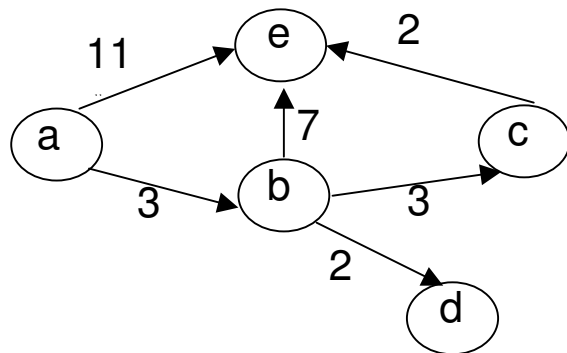
PROJETO DE ALGORITMOS PELA ESTRATÉGIA GULOSA

Agora vamos examinar o emprego da estratégia gulosa para desenvolver algoritmos para alguns **problemas típicos**.

Problema caminhos de custo mínimo em grafo orientado a partir de fonte

O problema consiste em determinar um **caminho de custo mínimo** a partir de um vértice fonte a cada vértice do grafo.

Consideremos um grafo orientado $G = \langle V, E \rangle$ com 5 vértices: $V = \{a, b, c, d, e\}$ e 8 arestas com a seguinte matriz de custos:



custo	a	b	c	d	e
a		3	∞	∞	11
b	∞		3	2	7
c	∞	∞		∞	2
d	∞	∞	∞		∞
e	∞	∞	∞	∞	

A fonte v_0 é o vértice a . Assim, inicialmente o conjunto I de vértices intermediários é vazio, e o vetor de distâncias é a primeira linha da matriz de custos acima, ou seja:

b	c	d	e
3	∞	∞	11

Em **cada passo**, seleciona-se um vértice que tenha distância mínima como novo intermediário, e são atualizadas as distâncias.

Assim, obtém-se a seguinte seqüência de passos:

passo	I	vértice	b	c	d	e
1	$\emptyset$	1: b	3	∞	5	10
2	{b}	2: d	3	6	5	10
3	{b, d}	3: c	3	6	5	8
4	{b, c, d}	4: e	3	6	5	8

Após o passo 4, teremos $I = \{b, c, d, e\}$, sendo os custos mínimos de caminhos da fonte a cada vértice alcançável dados pela última linha, ou seja:

b	c	d	e
3	6	5	8

Vamos construir incrementalmente um conjunto I de vértices e os caminhos a partir da fonte de custo mínimo, usando como intermediários apenas vértices no conjunto I .

Em **cada passo**, seleciona-se um novo vértice com distância mínima e são atualizados os custos mínimos a partir da fonte.

Quando $|I|=n$, teremos todos os caminhos com custo mínimo a partir da fonte.

Assim, podemos obter o seguinte algoritmo (onde a variável p mantém o conjunto de vértices ainda a considerar como possíveis intermediários).

Algoritmo: Custo mínimo de caminhos a partir de fonte em grafo orientado

Função $\text{Dist_fnt}(d:D) \rightarrow R$ {distância a partir da fonte em grafo}

{Entrada: grafo orientado valorado G com fonte v_0 e matriz custo: M (de custos)}

Saída: vetor $\text{dist}:V'$ (custos dos melhores caminhos a partir de v_0)}

sorte $D := \text{Grafos valorados com fonte}; R := V'; M := \underline{\text{matriz}}[0..n, 0..n] \underline{\text{de}} \mathbb{R}^+;$
 $V' := \underline{\text{vetor}}[1..n] \underline{\text{de}} \mathbb{R}^+; Vrt := \text{Vértices}; P := \text{Conj}[Vrt] \text{ {conjuntos de vértices}};$
var {auxiliares} $p:P$ {parte da entrada}; $e:Vrt$ {seleção};
 $V_0:Vrt$ {vértices não fonte}; $c:\mathbb{R}^+$ {distância};

0. $V_0 \leftarrow V - \{v_0\};$ {vértices não fonte: $v_1, \dots, v_n$ }
1. $p \leftarrow V_0;$ {inicializa porção da entrada com $\{v_1, \dots, v_n\}$ }
2. para i de 1 até n faça $\text{dist}[i] \leftarrow \text{custo}[0, i];$ {inicializa resposta parcial}
3. para i de 1 até n faça {itera: 10}
4. $e \leftarrow \text{vértice } v_j \in p \text{ com } \text{dist}[j] \text{ mínimo};$ {seleciona novo vértice}
5. $p \leftarrow p - \{e\};$ {remove vértice selecionado}
6. para cada $v_i \in V_0$ faça {vértices $v_1, \dots, v_n$: 9}
7. $c \leftarrow \text{dist}[j] + \text{custo}[v_j, v_i];$ {distância usando v_j }
8. se $c \leq \text{dist}[i]$ então $\text{dist}[i] \leftarrow c;$ {atualiza distância}
9. fim-para {6: cada $v_i \in V_0$ }
10. fim-para {3: repita}
11. retorne-saída(dist); {dá como saída resposta pronta}
12. fim-Função {fim do algoritmo Dist_fnt: distância a partir da fonte}

Pode-se determinar um **caminho de custo mínimo** a partir da fonte, guardando-se os valores de e , obtidos na linha 4 do algoritmo.

Podemos ver que esse algoritmo `Dist_fnt` foi obtido pela estratégia gulosa observando o que segue:

- as linhas de 0 a 2 fazem a inicialização: as linhas 0 e 1 preparam a porção da entrada e a linha 2 inicializa a saída;
- a iteração está nas linhas de 3 a 10: a linha 4 seleciona um elemento, que é removido na linha 5, sendo incluído na resposta parcial se viável nas linhas de 6 a 9;
- a finalização está na linha 11, quando se recupera a solução.

Problema de árvore geradora (ou de espalhamento) mínima

Uma **árvore geradora** (ou de espalhamento) de um grafo G é um subgrafo acíclico (i. e. sem ciclos) que contém todos os vértices (nodos) do grafo.

Se o grafo não é conexo, cada componente terá uma árvore geradora.

Uma **estratégia** sugerida pelo método guloso para determinar uma árvore geradora de custo mínimo é a seguinte:

As arestas serão consideradas em ordem não decrescente de seus custos, e vamos construir uma floresta de árvores.

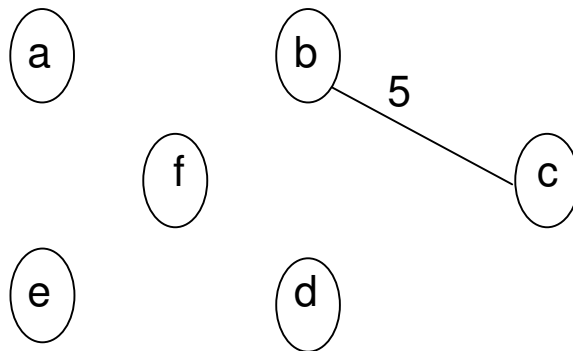
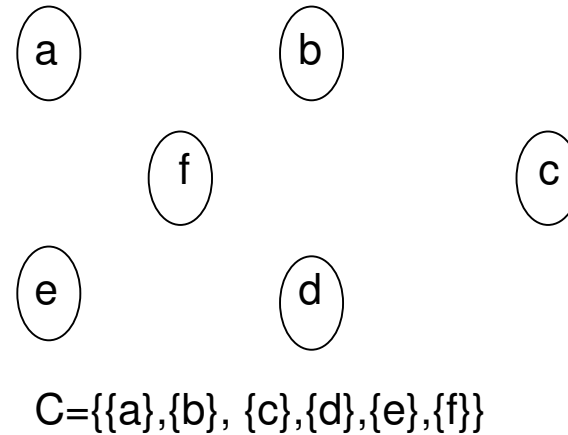
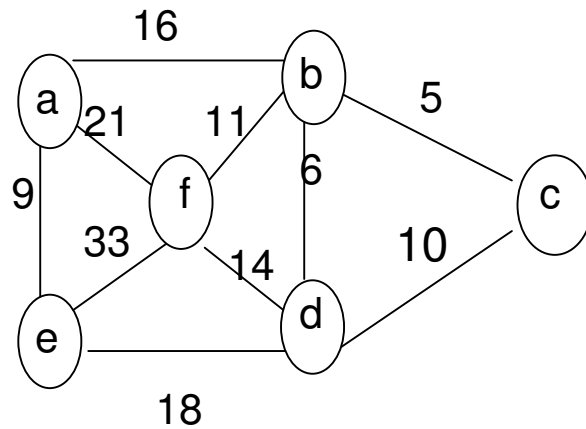
A cada passo, seleciona-se uma **aresta de custo mínimo** dentre as ainda não examinadas. Se sua inclusão criar um ciclo, ela é descartada; caso contrário, ela é incluída.

Consideremos um grafo não orientado $G = \langle V, E \rangle$ com

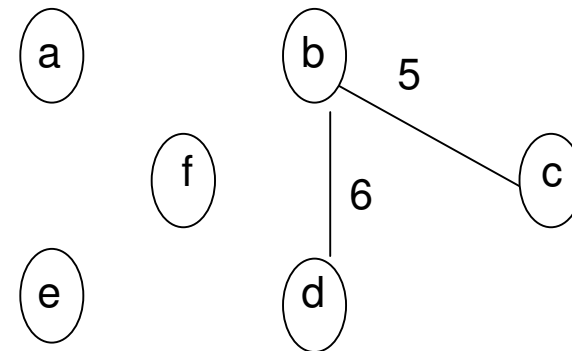
- 6 vértices: $V = \{a, b, c, d, e, f\}$;
- 10 arestas: $E = \{(a, b), (a, e), (a, f), (b, c), (b, d), (b, f), (c, d), (d, e), (d, f), (e, f)\}$;
- custos:

custo(a, b) = 16,	custo(a, e) = 9,	custo(a, f) = 21,
custo(b, c) = 5,	custo(b, d) = 6,	custo(b, f) = 11,
custo(c, d) = 10,	custo(d, e) = 18,	custo(d, f) = 14,
custo(e, f) = 33.		

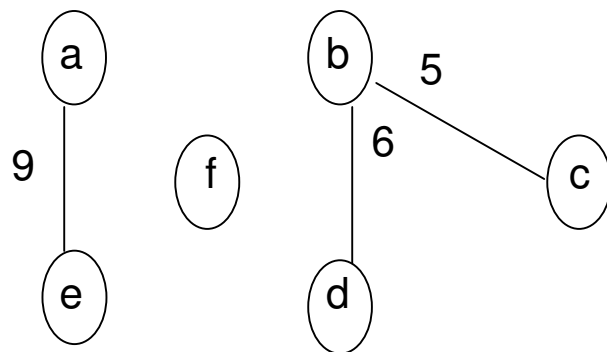
Uma **árvore espalhada mínima** para G pode ser construída de acordo com a estratégia acima.



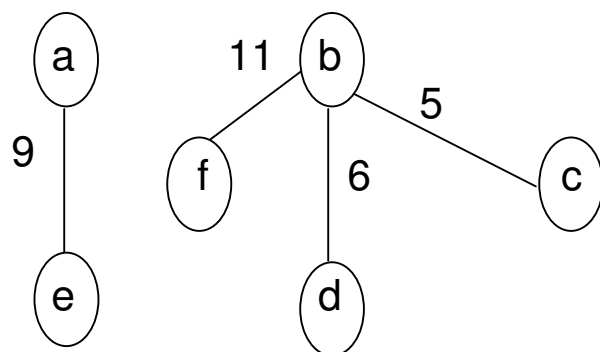
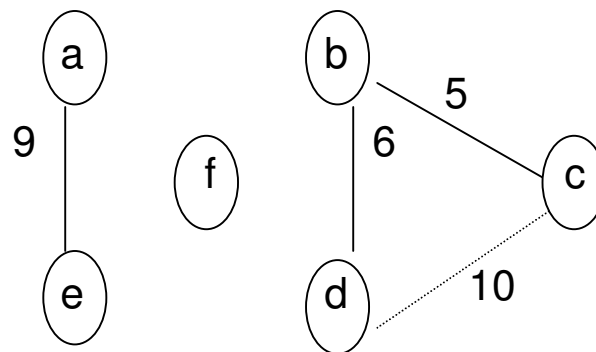
$$C = \{\{a\}, \{b, c\}, \{d\}, \{e\}, \{f\}\}$$



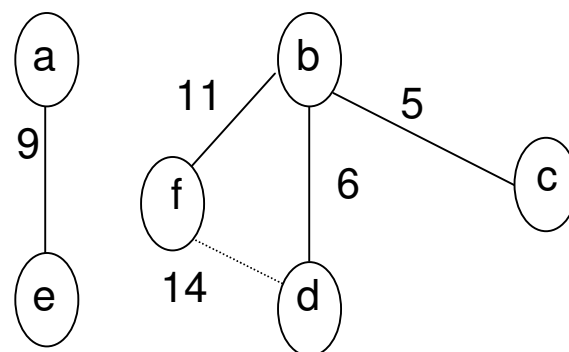
$$C = \{\{a\}, \{b, c, d\}, \{e\}, \{f\}\}$$

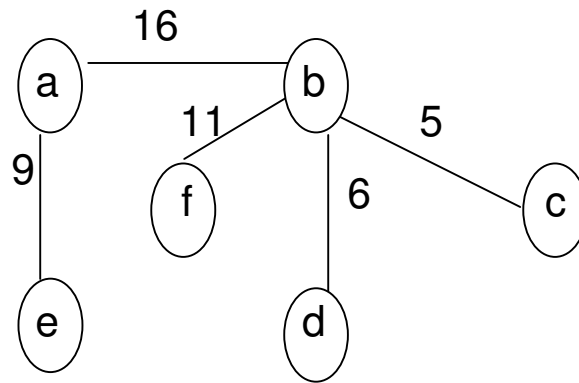


$$C = \{\{a, e\}, \{b, c, d\}, \{f\}\}$$



$$C = \{\{a, e\}, \{b, c, d, f\}\}$$





$C = \{\{a, e, b, c, d, f\}\}$

Construção de árvore espalhada mínima

Agora, vejamos como essas idéias levam ao projeto de um algoritmo para esse problema de árvore geradora (ou de espalhamento) mínima para um **grafo** (não orientado) pela estratégia gulosa.

A estratégia gulosa sugere selecionar arestas em **ordem crescente de custos**.

Construir um floresta usando as arestas dadas: cada aresta selecionada será incluída se não originar ciclo, podendo levar à fusão de componentes. Quando a floresta só tiver único componente, teremos uma **árvore geradora mínima**.

Algoritmo: Árvore geradora mínima de grafo

Função Árv_ger_mín($d:D$) $\rightarrow R$ {árvore geradora mínima de grafo conexo}

{*Entrada*: grafo não orientado (conexo) $d:D$ com m arestas valoradas}

Saída: árvore geradora mínima $r:R$ do grafo (conexo) $d:D$ }

sorte $D :=$ Grafos valorados; $R :=$ Árvores (valoradas); $E :=$ Arestas valoradas;

$P :=$ Filas de arestas; $S :=$ Florestas de árvores;

var $p:P$ {parte da entrada}; $s:S$ {parte da saída}; $e:E$ {elemento selecionado};

1. $p \leftarrow \langle e_1, \dots, e_m \rangle$; {fila com arestas em ordem de custos}
2. $s \leftarrow \{ \langle \{v\}, \emptyset \rangle / v \in V \}$; {floresta com componentes unitários}
3. repita {itera até $|s.V| = 1: 9$ }
4. $(u, v) \leftarrow \text{prim}(p)$; {seleciona aresta com custo mínimo}
5. $p \leftarrow \text{rem_P}((u, v), p)$; {remove aresta selecionada da fila}
6. $A' \leftarrow \text{comp}(u, s)$; $A'' \leftarrow \text{comp}(v, s)$; {componentes dos vértices}
7. se $A' \neq A''$ {testa criação de ciclo}
8. então substitua em s : $A' \notin A''$ por $A' \cup A''$; {inclui aresta}
9. até-que $|s.V| = 1$; {3: floresta com um único componente}
10. $r \leftarrow \text{árv}(s)$; {recupera árvore de floresta pronta}
11. retorne-saída(r); {dá saída r }
12. fim-Função {fim do algoritmo Árv_ger_mín: de grafo conexo}

Note que esse algoritmo `Árv_ger_mín` consegue determinar uma árvore geradora mínima (de um grafo conexo) sem ter necessidade de examinar todas as suas arestas. Essa é a razão de se utilizar uma iteração indefinida.

- as linhas 1 e 2 fazem a **inicialização**: a linha 1 prepara a porção da entrada (ordenada) e a linha 2 inicializa a saída;
- a **iteração** está nas linhas de 3 a 9: a linha 4 seleciona um elemento, que é removido na linha 5, sua viabilidade de estar ou não na solução é testada na linha 7, sendo incluído na resposta parcial se viável na linha de 8;
- a **finalização** está na linha 10, quando se recupera a solução.

Vimos três exemplos de algoritmos desenvolvidos pela estratégia gulosa

Cada um desses três algoritmos gulosos tem a estrutura geral de seqüência

Inicialização ; Iteração ; Finalização.

No algoritmo `Dist_fnt`, a iteração (nas linhas de 3 a 10) é definida: para ... faça Nos outros dois primeiros algoritmos (`Interc_Suc_Lst` e `Árv_ger_mín`) a iteração principal é indefinida: repita ... até-que Mas, já sabemos que uma iteração definida pode ser simulada por uma iteração indefinida (usando um contador como variável de controle).

COMPLEXIDADE DE ALGORITMOS GULOSOS

Complexidade de distância de fonte em grafo orientado

Considere o algoritmo `Dist_fnt` para distância (custo mínimo de caminhos) a partir de fonte em grafo orientado.

Como só é usada a operação de **retirada de elemento do conjunto**, pode-se usar uma representação com complexidade de retirada constante (linhas 0 e 1).

A linha 2 é executada n vezes. Assim, a complexidade da inicialização (linhas 0, 1 e 2) tem ordem **$O(n)$** .

A complexidade da linha 4 é o tamanho de p , que começa com n ($n = |V_0|$) e diminui de 1 a cada iteração.

A iteração das linhas de 3 a 10 é executada n vezes. O trecho das linhas 4 e 5 tem complexidade n . Assim, o corpo da iteração (linhas de 4 a 9) tem complexidade $|p| + n$, que varia durante a iteração ($|p| = n - i$, ou seja $(n - i) + n$).

Logo, a complexidade da iteração é dada pela somatória.

$$\sum_{i=0}^n [(n-i) + n] = O(n + n^2). \text{ Assim, } c_p^{\leq} [\text{Dist_fnt}](n) = O(n + n^2) = O(n^2).$$

Agora, vamos fazer uma **análise** mais cuidadosa e detalhada, usando as idéias vistas na seção 3.2 do capítulo 3 (cf. 3.2.2, 3.2.4 e 3.2.5).

Esse algoritmo `Dist_fnt` recebe como **entrada** $d:D$

um grafo orientado valorado G com fonte v_0 e matriz de custos

e fornece como **saída**

um vetor `dist` (com os custos dos melhores caminhos a partir de v_0).

Consideremos um grafo orientado G , com conjunto $V = \{v_0, v_1, \dots, v_n\}$ de vértices. Tomemos, como é natural, para tamanho da entrada o número n de vértices não fonte.

Nesse caso, parece natural tomar como **operações fundamentais** as manipulações com conjuntos (de vértices) e matrizes.

Como o algoritmo Dist_fnt é uma seqüência, podemos aplicar os resultados vistos na seção 3.2 (cf. 3.2.2).

O desempenho de Dist_fnt sobre uma entrada $d:D$ tem contribuições dadas por suas componentes: inicialização, iteração e finalização.

Para determinar essas contribuições, devemos obter os valores das entradas de cada uma dessas três componentes.

A inicialização (linhas de 0 a 2) fornece valores iniciais às variáveis:

- $p:P$ {parte da entrada}, $p_0 := \{v_1, \dots, v_n\}$ {tamanho $|p_0| = n$ }, e
- $\text{dist}:V'$ {custos dos melhores a partiacaminhos de v_0 },
 $\text{dist}_0 := [\text{custo}[0, 1], \dots, \text{custo}[0, n]]$ {tamanho $\dim(\text{dist}_0) = n$ }

Assim, a saída da inicialização é $\text{Incz_Dist_fnt}(G) = (p_0, \text{dist}_0)$.

O desempenho da inicialização tem as seguintes contribuições:

comando	desempenho
0. $V_0 \leftarrow V - \{v_0\};$	1
1. $p \leftarrow v_0;$	n
2. <u>para</u> i <u>de</u> 1 <u>até</u> n <u>faça</u> $\text{dist}[i] \leftarrow \text{custo}[0, i];$	n

Assim, temos:

$$\text{desemp}[\text{Incz_Dist_fnt}](G) = 1 + n + n \cdot n = 2 \cdot n + 1.$$

Agora, a iteração para i de 1 até n (linhas de 3 a 10) repete n vezes o corpo, selecionando um elemento, removendo-o e incluindo-o na resposta parcial se viável (linhas de 4 a 9).

São gerados novos valores para as variáveis $p:P$ e $\text{dist}:V'$ através de valores da variável $e:E$ {seleção}

$e:E$	$e_1 \in p_0$	...	$e_k \in p_{k-1}$	...	$e_n \in p_{n-1}$
$p:P$	$p_1 := p_0 - \{e_1\}$		$p_k := p_{k-1} - \{e_k\}$		$p_n = p_{n-1} - \{e_n\}$
$\text{dist}:V'$	dist_1	...	dist_k	...	dist_n

Assim, os valores sucessivos das variáveis $p:P$ e $\text{dist}:V$ têm tamanhos como se segue:

l	0	1	$\dots$	k	$\dots$	n
$ p_i $	n	$n-1$	$\dots$	$n-k$	$\dots$	0
$\text{Dim}(\text{dist}_i)$	n	n	$\dots$	n	$\dots$	n

Ao início da $i^{\text{ésima}}$ iteração, as variáveis $p:P$ têm tamanhos $|p_i| = n-i+1$ e $\text{dist}:V$ têm tamanhos $\text{dim}(\text{dist}_i) = n$.

Agora, o desempenho do corpo **Crp_Dist_fnt** (linhas de 4 a 9) em cada iteração é dado pela soma das contribuições das linhas de 4 a 5 e das linhas de 6 a 9.

Na $i^{\text{ésima}}$ iteração, as linhas 4 e 5 atuam sobre variáveis $p:P$ e $\text{dist}:V$ com tamanhos $|p_i| = n-i+1$ e $\text{dim}(\text{dist}_i) = n$. Assim, temos as seguintes cotas superiores para o desempenho das linhas 4 e 5:

Comando	cota superior
4. $e \leftarrow \text{vértice } v_j \in p \text{ com } \text{dist}[j] \text{ mínimo};$	$n-i+1$
5. $p \leftarrow p - \{e\};$	$n-i+1$

Assim, $\text{desemp}[\text{linhas 4-5}](p_i, \text{dist}_i) \leq (n-i+1) + (n-i+1) \leq 2 \cdot (n+1)$.

Na $i^{\text{ésima}}$ iteração, as linhas de 6 a 9 atuam sobre variáveis $p:P$ e $\text{dist}:V$ com tamanhos $|p_{i+1}| = n-i$ e $\text{dim}(\text{dist}_i) = n$.

Assim, temos as seguintes **cotas superiores** para desempenho das linhas 7 e 8:

comando	cota superior
7. $c \leftarrow \text{dist}[j] + \text{custo}[v_j, v_i];$	1
8. <u>se</u> $c \leq \text{dist}[i]$ <u>então</u> $\text{dist}[i] \leftarrow c$	1
linhas 7-8	$1 + 1 = 2$

Como na $i^{\text{ésima}}$ iteração, as linhas 7 e 8 são executadas com $n = |V_0|$ vezes, temos:

$$\text{desemp}[\text{linhas 6-9}](p_{i+1}, \text{dist}_i) \leq n \cdot 2 = 2 \cdot n.$$

Uma **cota superior** para o desempenho do corpo Crp_Dist_fnt na $i^{\text{ésima}}$ iteração é dado pela soma das cotas para as contribuições conforme a tabela seguinte:

$\text{desemp}[\text{linhas 4-5}](p_i, \text{dist}_i)$	$\leq$	$2 \cdot (n+1) = 2 \cdot n + 2$
+		
$\text{desemp}[\text{linhas 6-9}](p_{i+1}, \text{dist}_i)$	$\leq$	$2 \cdot n$
$\text{desemp}[\text{Crp_Dist_fnt}](p_i, \text{dist}_i)$	$\leq$	$2 \cdot (n+1) + 2 \cdot n = 4 \cdot n + 2$

Agora, como a iteração `Iter_Dist_fnt` (linhas de 3 a 10) repete n vezes o corpo `Crp_Dist_fnt`, uma cota superior para o desempenho da iteração `Iter_Dist_fnt` é $n \cdot (4 \cdot n + 2) = 4 \cdot n^2 + 2 \cdot n$.

Assim, temos para o desempenho de `Iter_Dist_fnt`

$$desemp[Iter_Dist_fnt](l_0, dst_0) \leq 4 \cdot n^2 + 2 \cdot n.$$

Agora, o desempenho do algoritmo 5.1.3: `Dist_fnt` é a soma das duas contribuições:

$desemp[Incz_Dist_fnt](G)$ e $desemp[Iter_Dist_fnt][Incz_Dist_fnt](G)$:

$$\begin{array}{rcl} desemp[Incz_Dist_fnt](G) & = & 2 \cdot n + 1 \\ + & & \\ desemp[Iter_Dist_fnt](p_0, dist_0) & \leq & 4 \cdot n^2 + 2 \cdot n \\ \hline desemp[Dist_fnt](G) & \leq & 4 \cdot n^2 + 4 \cdot n + 1 \end{array}$$

Portanto, temos para o desempenho de `Dist_fnt`

$$desemp[Dist_fnt](G) \leq 4 \cdot n^2 + 4 \cdot n + 1.$$

Assim, para a complexidade pessimista de `Dist_fnt` temos

$$c_p^{\leq} [\text{Dist_fnt}](n) = O(4 \cdot n^2 + 4 \cdot n + 1) = O(n^2).$$

Logo, o algoritmo: `Dist_fnt` (para distância de fonte em grafo orientado) tem complexidade pessimista quadrática:

$$c_p^{\leq} [\text{Dist_fnt}](n) = O(n^2).$$

Complexidade de intercalação ótima de listas

Considere o algoritmo `Interc_Suc_Lst` (para intercalação ótima de listas).

O algoritmo efetua operações sobre o multiconjunto L , portanto a complexidade do algoritmo pode variar conforme a estrutura de dados escolhida para representar L .

Vamos considerar a representação do multiconjunto L por uma lista encadeada, em que cada elemento guarda tanto uma lista quanto seu tamanho.

- para determinar as duas menores listas, a lista é percorrida, com complexidade $O(n)$;

- **para remoção** a complexidade é $O(n)$;
- **para a inclusão** a complexidade é constante (pois, como um multiconjunto pode ter diversas ocorrências de um mesmo elemento, não se necessita verificar a repetição de elementos).

Assim, podemos estimar a complexidade pessimista do algoritmo Interc_Suc_Lst, desprezando a inicialização (linha 1) e a finalização (linhas 9 e 10), nos concentrando na iteração (linhas de 2 e 8).

A cada iteração do corpo (linhas de 3 e 7), o tamanho de L diminui de 1, logo a iteração é executada para $|L|$ variando de n a 1.

A **complexidade do corpo da iteração** tem ordem

$$O(n) + O(1) + O(m' + m'' - 1),$$

onde m' e m'' são os tamanhos das duas listas selecionadas.

A ordem da complexidade da iteração é dada pela somatória

$$\sum_{i=0}^{n-2} (n + m' + m'').$$

Mas $\sum_{i=0}^{n-2} n = O(n^2)$ e $\sum_{i=0}^{n-2} (m' + m'') = O(n^2 \cdot m)$, onde m é o máximo dos comprimentos das n listas em L (pois $m' + m'' \leq n \cdot m$).

Portanto, a **complexidade do algoritmo Interc_Suc_Lst** tem ordem

$$c_p^{\leq} [\text{Interc_Suc_Lst}](n) = O(n^2 + n^2 \cdot m) = O(n^2 \cdot m).$$

Agora, vamos fazer uma análise mais cuidadosa e detalhada, como antes usando as idéias vistas na seção 3.2 do capítulo 3.

Esse algoritmo Interc_Suc_Lst recebe como **entrada** $d:D$

multiconjunto L de n listas (com seus comprimentos)

e fornece como **saída**

o par (intercalação das n listas em L , número mínimo de comparações).

Nesse caso, parece natural tomar como **operação fundamental** a comparação entre elementos das listas.

Também, parece natural tomar como tamanho da entrada o par formado pelo tamanho n do multiconjunto L e pelo comprimento m da maior lista em L .

Como o algoritmo `Interc_Suc_Lst` é uma seqüência, seu desempenho sobre uma entrada $L:D$ tem contribuições dadas por suas componentes:

Inicialização, Iteração e Finalização.

A **inicialização** (linha 1) e a **finalização** (linhas 9 e 10) não envolvem comparações.

Assim:

$$desemp[Incz_Interc_Suc_Lst](L) = 0 \text{ e } desemp[Fnl_Interc_Suc_Lst](L) = 0.$$

Agora, examinemos a **iteração**:

repita até-que $|L| = 1$ (linhas de 2 a 8).

A cada **iteração** do corpo (linhas de 3 a 7), as duas menores listas em L são substituídas por sua intercalação, fazendo o tamanho de L diminuir de 1, logo a iteração é executada para $|L|$ variando de n a 1 : n vezes.

Realmente, os valores sucessivos da variável $L:D$ têm tamanhos como se segue

i	0	1	...	k	...	$n-1$
$ L_i $	n	$n-1$	...	$n-k$	...	1

Na i ésima **iteração**, a variável $L:D$ tem tamanho $|L_i| = n-i$. Assim, temos as seguintes cotas superiores para o desempenho das linhas de 3 e 7:

comando	cota superior
3. $(L', L'') \leftarrow 2 \text{ menores em } L;$	$n-i$
4. $L \leftarrow L - \{L', L''\};$	$n-i$
5. $L^* \leftarrow \text{Interc}(L', L'');$	$\text{compr}(L') + \text{compr}(L'') - 1$
6. $L \leftarrow L \cup \{L^*\};$	0
7. $\text{ncp} \leftarrow \text{ncp} + \text{compr}(L') + \text{compr}(L'') - 1;$	0

Assim, com m'_i e m''_i para os tamanhos das duas listas selecionadas na i ésima iteração, temos

$$\begin{aligned} \text{desemp}[\text{Crp_Interc_Suc_Lst}](L_i, \text{ncp}_i) &\leq n-i + n-i + m'_i + m''_i + 0, \text{ logo,} \\ \text{desemp}[\text{Crp_Interc_Suc_Lst}](L_i, \text{ncp}_i) &= 2 \cdot (n-i) + m'_i + m''_i \leq 2 \cdot n + m'_i + m''_i. \end{aligned}$$

Portanto, uma cota superior para o desempenho da iteração $\text{Iter_Interc_Suc_Lst}$ é a somatória das $n-1$ contribuições:

$$2 \cdot n + m'_0 + m''_0 + \dots + 2 \cdot n + m'_i + m''_i + \dots + 2 \cdot n + m'_{n-2} + m''_{n-2}$$

Essa **cota superior para a iteração** é:

$$\sum_{i=0}^{n-2} (2 \cdot n + m'_i + m''_i) = 2 \cdot \sum_{i=0}^{n-2} n + \sum_{i=0}^{n-2} (m'_i + m''_i).$$

Agora, $\sum_{i=0}^{n-2} n = (n-1) \cdot n$. Por outro lado, tomando m como o máximo dos comprimentos das n listas em L , temos (já que $m'_i + m''_i \leq n \cdot m$)

$$\sum_{i=0}^{n-2} (m'_i + m''_i) \leq \sum_{i=0}^{n-2} (n \cdot m) = \sum_{i=0}^{n-2} m = (n-1) \cdot n \cdot m.$$

Assim, temos para o **desempenho de Iter_Interc_Suc_Lst**

$$desemp[Iter_Interc_Suc_Lst](L) \leq (n-1) \cdot n + (n-1) \cdot n \cdot m.$$

Agora, o desempenho do algoritmo Interc_Suc_Lst é a soma das três contribuições devidas:

à inicialização, à iteração e à finalização.

Portanto, temos para o **desempenho de Interc_Suc_Lst**

$$desemp[Interc_Suc_Lst](L) \leq 0 + (n-1) \cdot n \cdot (1+m) + 0.$$

Assim, para a complexidade pessimista de Interc_Suc_Lst temos

$$c_p^{\leq}[\text{Interc_Suc_Lst}](n, m) = O((n-1) \cdot n \cdot (1+m)) = O(n^2 \cdot m).$$

Portanto, o algoritmo **Interc_Suc_Lst** (para intercalação ótima de listas) tem **complexidade pessimista**

$$c_p^{\leq}[\text{Interc_Suc_Lst}](n, m) = O(n^2 \cdot m).$$